

Hash Table Performance

Title: Hash Table Performance Benchmark: Chaining vs Open Addressing

Course: BSc EEE. Data Structure and Algorithms

Student: Veronica Kavata Kitetu

ID: FEE3-2883-2025

Date: 0706/2026

1. Introduction

This project implements and evaluates the performance of two hash table collision resolution techniques: **chaining** and **open addressing (linear probing)**. The implementations are written from scratch in C++ and compared against the standard library implementation `std::unordered_map`.

The main objective is to analyse insertion and search performance under different input conditions, including random and adversarial data designed to cause collisions.

2. Data Structures Used

2.1 Chaining Hash Table

Chaining resolves collisions by storing multiple key-value pairs in the same index using a linked list.

Each index in the hash table contains a list of entries.

Advantages:

- Simple collision handling
- Performs well under high collision rates

Disadvantages:

- Extra memory overhead due to linked lists
-

2.2 Open Addressing (Linear Probing)

Open addressing stores all elements directly inside the array. When a collision occurs, the algorithm searches for the next available slot.

Advantages:

- Memory efficient (no linked lists)
- Better cache performance

Disadvantages:

- Performance degrades due to clustering
-

3. Hash Function

A simple modulo-based hash function was used:

$$h(k) = k \bmod \text{table_size}$$

This function is efficient but can lead to collisions when keys are not well distributed.

4. Time Complexity Analysis

Operation	Chaining	Open Addressing
Insert	O(1) average, O(n) worst	O(1)average, O(n)worst

Search	$O(1)$ average, $O(n)$ worst	$O(1)$ average, $O(n)$ worst
--------	------------------------------	------------------------------

Explanation:

- The average case assumes a uniform distribution of keys
 - Worst case occurs when many keys collide into the same index
-

5. Space Complexity

- **Chaining:** $O(n + m)$, where m is the number of buckets
- **Open Addressing:** $O(m)$, fixed-size array

Open addressing uses less memory overhead but requires careful sizing to avoid clustering.

6. Experimental Setup

The program was tested using three implementations:

- Chaining hash table
- Open addressing hash table

- `std::unordered_map`

Input Sizes:

- 1,000 elements
- 10,000 elements
- 100,000 elements

Test Cases:

1. Random input keys
2. Adversarial input (designed to cause collisions)

7. Results and Observations

7.1 Random Input

Size	Chaining	Open Addressing	std::unordered_map
1,000	Fast	Very Fast	Fastest
10,000	Moderate	Fast	Fastest
100,000	Slow	Moderate	Fastest

7.2 Adversarial Input

Size	Chaining	Open Addressing	std::unordered_map
1,000	Moderate	Slow	Fastest
10,000	Slow	Very Slow	Fastest
100,000	Very Slow	Extremely Slow	Fastest

8. Discussion

From the results, it is observed that:

- `std::unordered_map` consistently performs the best due to optimised hashing and dynamic resizing.
 - Chaining handles collisions more gracefully than open addressing.
 - Open addressing suffers from clustering, especially under adversarial input.
 - Performance of both custom implementations degrades significantly in worst-case scenarios.
-

9. Conclusion

This project demonstrates that hash tables provide efficient average-case performance for insertion and lookup operations. However, performance heavily depends on the quality of the hash function and collision handling strategy.

Among the implemented methods:

- Chaining is more stable under collisions
- Open addressing is more memory efficient but less robust
- `std::unordered_map` outperforms both due to internal optimisations.